PracticAI

Python Syntax Every AI Engineer Should Know

Most AI engineers don't struggle with models.

They struggle with **clean Python code.**

(Swipe to learn →)

Naresh Edagotti
Follow For More

List Comprehensions

Instead of writing loops:

```
squares = []
for x in range(10):
    squares.append(x*x)
```

Use list comprehension:

```
squares = [x*x for x in range(10)]
```

MENTAL MODEL

```
[expression for item in iterable]
```

Transform each element.

When to Use

→ Transform lists

→ Feature preprocessing

→ Data transformations

Conditional List Comprehension

Filter elements while building lists.

```
nums = [1, 2, 3, 4, 5, 6]

evens = [x for x in nums if x % 2 == 0]
```

```
[2, 4, 6]
```

MENTAL MODEL

```
[expression for item in iterable if condition]
```

Filter + transform together.

When to Use

→ Filtering datasets

→ Cleaning data

→ Feature selection

Dictionary Comprehension

Create dictionaries dynamically.

```
nums = [1, 2, 3, 4]

squares = {x: x*x for x in nums}
```

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

MENTAL MODEL

```
{key:value for item in iterable}
```

Generate mappings.

When to Use

→ lookup tables

→ feature mappings

→ token → id mapping

Enumerate

Access index and value.

```
for i, val in enumerate(nums):
    print(i, val)
```

MENTAL MODEL

```
(index, value)
```

Track position while iterating.

When to Use

→ indexing datasets

→ processing batches

→ debugging loops

Zip

Iterate through multiple lists together.

```
names = ["Alice", "Bob"]
scores = [85, 92]

for name, score in zip(names, scores):
    print(name, score)
```

MENTAL MODEL

```
(list1[i], list2[i])
```

Combine sequences row-wise.

When to Use

→ features + labels

→ predictions + probabilities

→ combining datasets

DefaultDict

Avoid missing key errors.

```
from collections import defaultdict

freq = defaultdict(int)

for word in words:
    freq[word] += 1
```

MENTAL MODEL

```
missing key → auto initialize
```

Dictionary with default values.

When to Use

→ frequency counting

→ grouping data

→ NLP preprocessing

Counter

Count elements quickly.

```
from collections import Counter

nums = [1, 2, 2, 3, 3, 3]

Counter(nums)
```

Result

```
{3: 3, 2: 2, 1: 1}
```

MENTAL MODEL

```
element → frequency
```

Automatic frequency table.

When to Use

→ token frequency

→ word counts

→ dataset statistics

Sorting with Key

Sort objects with custom logic.

```
sorted(words, key=len)
```

MENTAL MODEL

```
sort_by(key_function)
```

Sort based on computed value.

When to Use

→ ranking predictions

→ sorting features

→ leaderboard metrics

Lambda Functions

Small anonymous functions.

```
squares = list(map(lambda x: x*x, nums))
```

MENTAL MODEL

```
lambda input : expression
```

One-line function.

When to Use

→ quick transformations

→ sorting keys

→ small data operations

Set Operations

Sets remove duplicates.

```
unique = set(nums)
```

Membership check:

```
if x in unique:
```

MENTAL MODEL

```
unique elements only
```

Fast lookup structure.

When to Use

→ removing duplicates

→ fast membership checks

→ filtering datasets

Unpacking

Assign multiple variables easily.

```
a, b = 10, 20
```

List unpacking:

```
first, *rest = [1, 2, 3, 4]
```

MENTAL MODEL

```
sequence → variables
```

Split structured data.

When to Use

→ model outputs

→ batch processing

→ returning multiple values

*args and **kwargs

Flexible function arguments.

```
def func(*args):
    print(args)
```

```
def func(**kwargs):
    print(kwargs)
```

MENTAL MODEL

```
*args → tuple
**kwargs → dictionary
```

Dynamic arguments.

When to Use

→ ML library APIs

→ flexible pipelines

→ configurable functions

Generator Expressions

Lazy evaluation.

```
squares = (x*x for x in range(1000000))
```

MENTAL MODEL

```
generate → consume → discard
```

Stream values instead of storing.

When to Use

→ large datasets

→ memory optimization

→ streaming pipelines

yield (Generators)

Create lazy iterators.

```
def numbers(n):
    for i in range(n):
        yield i
```

MENTAL MODEL

```
pause → return → resume
```

Function that produces values one-by-one.

When to Use

→ data loaders

→ streaming pipelines

→ batch generators

any() and all()

Quick boolean checks.

```
any(x > 3 for x in nums)
```

```
all(x > 0 for x in nums)
```

MENTAL MODEL

```
any → at least one True
all → all True
```

When to Use

→ validation checks

→ filtering logic

→ safety conditions

dict.get()

Safe dictionary access.

```
count[word] = count.get(word, 0) + 1
```

MENTAL MODEL

```
dict[key] else default
```

Fallback value.

When to Use

→ counting patterns

→ NLP tokenization

→ feature statistics

itertools

Powerful iteration utilities.

```
from itertools import combinations

list(combinations([1, 2, 3], 2))
```

MENTAL MODEL

```
generate combinations/permutations
```

When to Use

→ feature combinations

→ sampling datasets

→ experimentation

Type Hints

Modern Python uses types.

```
def predict(text: str) → float:
    return model(text)
```

MENTAL MODEL

```
input_type → output_type
```

Self-documenting code.

When to Use

→ production ML systems

→ APIs

→ large codebases

Dataclasses

Simple structured objects.

```
from dataclasses import dataclass

@dataclass
class Prediction:
    text: str
    score: float
```

MENTAL MODEL

```
structured data container
```

When to Use

→ predictions

→ configs

→ structured outputs

Context Manager

Automatic resource management.

```
with open("data.txt") as f:
    data = f.read()
```

MENTAL MODEL

```
open → use → auto close
```

When to Use

→ file reading

→ model loading

→ dataset processing

Python is the Backbone of AI

Most AI systems aren't just models.

They are
Python pipelines.

The better your Python skills,
the easier it becomes to build:

- → **ML pipelines**
- → **GenAI applications**
- → **production AI systems**
- → **scalable AI infrastructure**